The Inverse Transform

Mark M. Fredrickson (mfredric@umich.edu)

Module 2: Generate

Computational Methods in Statistics and Data Science (DATASCI 406)

Review and Overview of Module 2

Module 1 Review

In Module 1, we used **pre-existing** sampling functions to

- Estimate integrals, expectations, and probabilities
- Find more efficient Monte Carlo estimators (**lower variance**)
- Compare estimators for parameters on **bias**, **variance**, **mean squared error**.
- Construct hypothesis tests using **null and alternative distributions**, **compute** p-**values**, **invert** into a confidence interval
- Evaluate confidence intervals for **coverage** and **expected width**

Module 2 Overview

In this module we **create our own sampling functions**

- Turn uniform draws into **any** distribution whose quantile function we can write
- **Transform** and **mix** variables we can already draw
- Sample a density we can only **evaluate**, by **rejecting** or **reweighting**
- **Factor** a joint distribution, draw one coordinate at a time
- Build a **Markov chain** whose long run behavior is the distribution we want

Why generate random numbers?

Our data are already random. Why manufacture more?

- Monte Carlo **integration**, **estimation**, **hypothesis tests** (module 1)
- **Resampling**: bootstrap, permutation tests, cross validation (module 3)
- Non-deterministic **optimization** (module 4)
- **Random assignment** in trials, **random sampling** from a population
- **Simulating** a process to see what it produces
- **Noise** for privacy, **cryptography**

Each needs a sampler for a particular distribution.

Pseudorandom Numbers

Random and Pseudorandom Numbers

When we used `runif`, we claimed this generated **random variables uniformly distributed**.

An event is **random** if we cannot predict it, knowing everything we could know.

$$\underbrace{f(x) = a + bx}_{\text{deterministic}} \qquad \underbrace{f(x) = a + bx + \epsilon,\ \epsilon \sim N(0,1)}_{\text{random}}$$

True randomness needs a physical source: atmospheric noise, thermal noise, radioactive decay.

Physical sources are slow, and they **cannot be reproduced**.

Pseudorandom numbers are deterministic, but close enough that no test we run tells the difference.

Where would you get true randomness?

```
library(random)                    # random.org, atmospheric noise
randomNumbers(10, 0, 255)

runif(10, 0, 255)                  # pseudorandom
```

The entropy pool, in base R

macOS and Linux expose the OS entropy pool as a file. **Windows does not.**

```
> con <- file("/dev/urandom", "rb", raw = TRUE)
> V <- readBin(con, "integer", n = 6, size = 2, signed = FALSE)
> close(con)
> V
[1] 38738 60890 56231 65318 11557 47707
> V/2^16
[1] 0.5910950 0.9291077 0.8580170 0.9966736 0.1763458 0.7279510
```

Each V is 16 random bits, so $V/2^{16}$ is our uniform. Code in `01-entropy-pool.R`.

Comparison

Source	Per call	Reproducible?
random.org, over the network	1.7 s	no
`/dev/urandom`, the OS entropy pool	0.16 ms	no
`runif`	0.005 ms	**yes**

Speed matters, but one big feature is **reproducibility.** We can get the same random sequence by setting a "seed" value.

Uniform bits

Computers represent numbers as groups of **bits**: values that are 0 or 1.

Generating a **(pseudorandom) number** is equivalent to producing random bits.

A uniform distribution on the integers 0 to $2^n - 1$ needs n IID bits with

$$P(B_i = 1) = P(B_i = 0) = 0.5$$

Let V be the number encoded by those n bits, then

$$U = \frac{V}{2^n} \sim \text{Uniform}(0, 1)$$

Pseudorandom Number Generators (PRNGs)

Computers are **deterministic** so how do we get "random numbers?"

A **PRNG** takes a **seed** and returns a number and a **new seed**:

```
prng <- function(seed) {
  # do something
  c(random_number, new_seed)
}
```

Nothing here is random, but the sequence looks **as if** it were $U(0, 1)$.

The seed is what makes it reproducible: `set.seed()` picks where you start.

A very simple generator

A **linear congruential generator**:

```
lcg <- function(seed) {
  a <- 5; c <- 1; m <- 8
  v <- (a * seed + c) %% m
  c(v/m, v) # random number, next seed in sequence
}
```

LCG sequence

```
rngs <- numeric(16)
seed <- 7
for (i in 1:16) {
  rs <- lcg(seed)
  rngs[i] <- rs[1]
  seed <- rs[2]
}
rngs
```

```
##  [1] 0.500 0.625 0.250 0.375 0.000 0.125 0.750 0.875 0.500
## [10] 0.625 0.250 0.375 0.000 0.125 0.750 0.875
```

Evaluating PRNGs

This was not a great choice of a PRNG (but it was fast!)

- The sequence **repeats after 8 values**
- Each value fixes the next, so the **period** cannot exceed the modulus m
- After one cycle the draws are no longer plausibly independent

How would we know a generator is good?

- We **cannot prove** it. No finite test certifies randomness.
- We **can reject** violations, with the hypothesis tests like module 1 built
- Packages `TestU01` and `diehard` run hundreds of them

PRNGs in R

R implements several; see `?Random`.

The default **Mersenne-Twister** has period $2^{19937} - 1$ (the LCG had 2^3).

```
set.seed(3949392) # fixes the starting point
runif(3)
```

```
## [1] 0.8447 0.9390 0.2587
```

From here we assume `runif()` works, and ask what we can build from it.

From Uniform to Anything

Distribution and quantile functions

The **(cumulative) distribution function** of a random variable is

$$F_X(t) = \Pr(X \leq t).$$

It is **non-decreasing** and **right continuous**, with $0 \leq F_X(t) \leq 1$.

That lets us define the **quantile function**

$$Q_X(p) = \inf \{x : F_X(x) \geq p\}, \quad p \in [0, 1].$$

When F_X is continuous and strictly increasing, this is just F_X^{-1}.

Example: uniform on $(0, \theta)$

For $X \sim U(0, \theta)$ with $f(x) = 1/\theta$ on $(0, \theta)$:

$$F(t) = \int_0^t \frac{1}{\theta}\, dx = \frac{t}{\theta}.$$

X is continuous, so invert directly:

$$u = \frac{t}{\theta} \quad \Rightarrow \quad Q(u) = \theta u.$$

Notice: When we sampled `50 * runif(n)` to get $U(0, 50)$ that is the same as $Q(U)$ where $U \sim U(0, 1)$. We'll prove the general case of this shortly.

The inverse transform method

Let X be a **continuous** random variable with quantile function Q_X.

For $U \sim \text{Uniform}(0, 1)$,

$$Y = Q_X(U)$$

has the same distribution as X.

One uniform draw, one function evaluation, one sample from X.

Proof

Since X is continuous, $\inf\{x : F(x) \geq u\} = \inf\{x : F(x) = u\}$, so $F(\inf\{x : F(x) = u\}) = u$.

And $F_U(u) = u$.

$$
\begin{aligned}
\Pr(Q(U) \leq x) &= \Pr(\inf\{x : F(x) \geq U\} \leq x) && \text{(definition)} \\
&= \Pr(\inf\{x : F(x) = U\} \leq x) && \text{(continuity)} \\
&= \Pr(F(\inf\{x : F(x) = U\}) \leq F(x)) && (F \text{ non-decreasing}) \\
&= \Pr(U \leq F(x)) && \text{(first fact)} \\
&= F(x) = \Pr(X \leq x) && \text{(second fact)}
\end{aligned}
$$

Corollary: the probability integral transform

Run the argument backwards. For continuous X,

$$F_X(X) \sim U(0,1).$$

Feeding a random variable into **its own** CDF gives a uniform.

The inverse transform takes uniforms to X; this takes X back to uniforms.

We use it to **test** samplers, and again in module 3.

Example: the exponential

$$f(x) = \lambda e^{-\lambda x}, \quad x \geq 0, \quad \lambda > 0$$

$$F(x) = \int_0^x \lambda e^{-\lambda t}\, dt = 1 - e^{-\lambda x}$$

Solve $u = 1 - e^{-\lambda x}$ for x:

$$Q(u) = -\frac{\log(1-u)}{\lambda}$$

If $U \sim U(0,1)$ then $1 - U \sim U(0,1)$, so $-\log(U)/\lambda$ works too.

Implementing it

```
rexp_ours <- function(n, rate = 1) {
  -log(runif(n)) / rate
}

x <- rexp_ours(100)
```

Checking your work: QQ plots

Quantile functions **uniquely determine** a distribution.

F_X and F_Y may live on different scales, but Q_X and Q_Y both map $(0, 1) \to \mathbb{R}$.

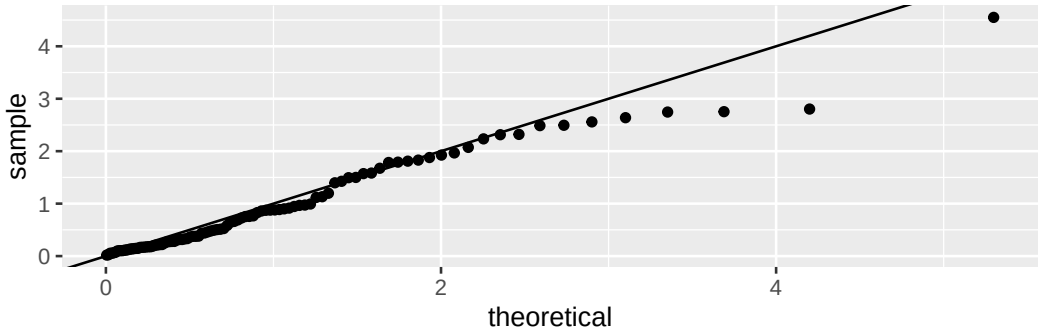
A **QQ plot** draws $(Q_X(u), Q_Y(u))$ for many $u \in (0, 1)$.

If the distributions agree, the points land on the 45 degree line.

Gives visual test of correctness.

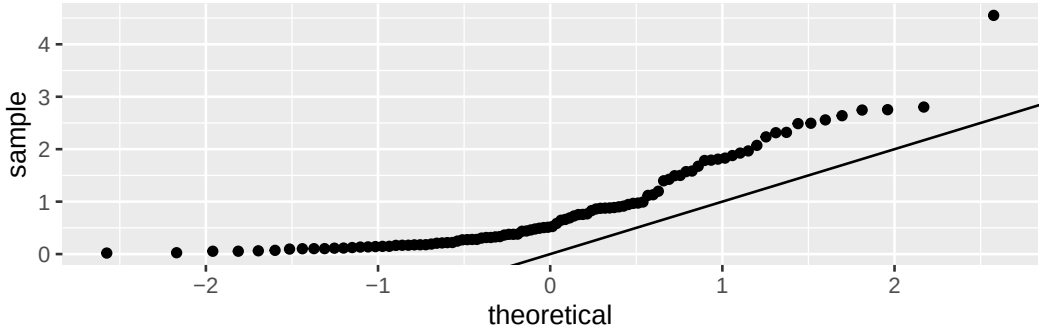
QQ plot: right distribution

```
ggplot(data.frame(sample = x), aes(sample = sample)) +
  geom_qq(distribution = qexp) +
  geom_abline(intercept = 0, slope = 1)
```



QQ plot: wrong distribution

```
ggplot(data.frame(sample = x), aes(sample = sample)) +
  geom_qq(distribution = qnorm) +
  geom_abline(intercept = 0, slope = 1)
```

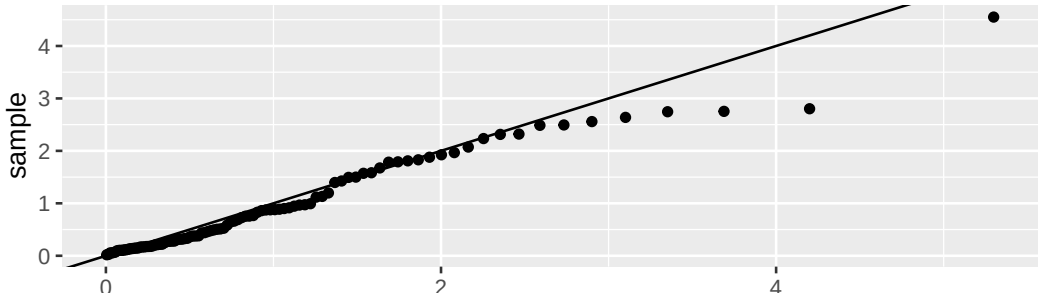


When R has no q function

geom_qq takes **any** quantile function, including one we wrote ourselves.

```r
q_ours <- function(p, rate = 1) -log(1 - p) / rate

ggplot(data.frame(sample = x), aes(sample = sample)) +
  geom_qq(distribution = q_ours) +
  geom_abline(intercept = 0, slope = 1)
```



Discrete Random Variables

Why the proof breaks

The continuous proof used continuity of F to write

$$Q(u) = \inf\{x : F(x) \geq u\} = \inf\{x : F(x) = u\}.$$

There had to be **some** point with $F(x) = u$.

For discrete X there need not be: if $X \sim \text{Bin}(1, 0.5)$, no x has $F(x) = 0.75$.

$Q(u)$ is still well defined. It is a **step function**.

We need a different argument and a different algorithm.

Q as a step function

For any discrete X,

$$Q(u) = \min\{x : F(x) \geq u\} = \min\left\{x : \sum_{i \leq x} p(i) \geq u\right\}$$

So if $Q(u) = x$, then $\sum_{i \leq x} p(i) \geq u$ and $\sum_{i \leq x-1} p(i) < u$.

$$Q(u) = x \quad \text{such that} \quad F(x-1) < u \leq F(x).$$

The uniform lands in one of the **jumps** of F, and that jump names the value.

The algorithm

Q is a step function, so instead of building, tabulate probabilities until you pass u:

1. Generate $U \sim U(0,1)$, set $v = 0$
2. For $x_1 < x_2 < \ldots$:
   a. $v \leftarrow v + p(x_i)$
   b. if $v \geq u$, return x_i

Works for **any** discrete distribution with a computable PMF.

Benford's Law, revisited

Recall, Benford's Law describes the leading digit D for random variables that are products of independent components:

$$\Pr(D = d) = \log_{10}\left(\frac{d+1}{d}\right), \quad d = 1, \ldots, 9.$$

```
##                1     2     3     4     5     6     7     8
## P(D = d) 0.301 0.176 0.125 0.097 0.079 0.067 0.058 0.051
## F(d)     0.301 0.477 0.602 0.699 0.778 0.845 0.903 0.954
##                9
## P(D = d) 0.046
## F(d)     1.000
```

$F(d)$ climbs to 1. A uniform draw lands in exactly one of those steps.

Closed form CDF for Benford

While we often have to resort to $F(x) = \sum_{v \leq x} P(X = v)$ for discrete CDFs, sometimes they have **closed form expressions**.

For example, Benford:

$$\Pr(D \leq d) = \sum_{i=1}^{d} \log_{10}\left(\frac{i+1}{i}\right) = \log_{10}\left(\frac{2 \times 3 \times \cdots \times (d+1)}{1 \times 2 \times \cdots \times d}\right) = \log_{10}(d+1)$$

The quantile function of D

This means we also get a closed form quantile function. Apply $F(d-1) < u \leq F(d)$ with $F(d) = \log_{10}(d+1)$:

$$\begin{aligned} Q(u) &= d \text{ s.t. } \log_{10}(d) < u \leq \log_{10}(d+1) \\ &= d \text{ s.t. } d < 10^u \leq d+1 \\ &= d \text{ s.t. } d-1 < 10^u - 1 \leq d \\ &= \lceil 10^u - 1 \rceil \end{aligned}$$

As before, draw $U \sim U(0,1)$ and apply $Q(U)$.

Sampling from D

```
rbenford <- function(n) ceiling(10^runif(n) - 1)

k <- 10000
rbind(analytic  = log10((2:10) / (1:9)),
      empirical = table(rbenford(k)) / k)
```

```
##                1      2      3       4       5       6
## analytic  0.3010 0.1761 0.1249 0.09691 0.07918 0.06695
## empirical 0.3115 0.1709 0.1259 0.09380 0.07780 0.06650
##                7       8       9
## analytic  0.05799 0.05115 0.04576
## empirical 0.05230 0.05250 0.04880
```

Compare `10^runif(n)` to $x = 10^m \cdot 10^f$ from the module 1 exercise.

Summary

- A PRNG gives **uniform** draws. Everything else is built on them.
- $Q_X(p) = \inf\{x : F_X(x) \geq p\}$ is defined for **every** distribution
- **Continuous**: $Q_X(U) \sim X$. One uniform, one function call.
- **Corollary**: $F_X(X) \sim U(0, 1)$, the probability integral transform
- **Discrete**: Q is a step function. Walk the PMF, or find Q in closed form.
- **Check every sampler with a QQ plot**

Next: what to do when we cannot write Q down.